EXPERIMENT 8: HADOOP AND MAPREDUCE

Single-machine Ubuntu practical guide — commands and what each step does

Aim: Install Hadoop, configure HDFS and YARN, and execute a MapReduce Word Count program.

How to use this guide: Run commands one at a time in Ubuntu Terminal. For the XML steps, paste the whole block, including the final EOF line. Verify each step before continuing.

Important: This is a fresh single-machine setup. Do not format an HDFS installation that already contains data.

STEP 1 — INSTALL JAVA AND SSH

What this does: Java runs Hadoop. SSH allows Hadoop to start its local services.

```
sudo apt update
sudo apt install openjdk-11-jdk openssh-server wget -y
java -version
```

STEP 2 — DOWNLOAD AND EXTRACT HADOOP

What this does: Downloads Hadoop 3.3.6, extracts it, and places it in /usr/local/hadoop.

```
cd ~/Downloads
wget https://archive.apache.org/dist/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R "$USER":"$USER" /usr/local/hadoop
```

STEP 3 — CONFIGURE ENVIRONMENT VARIABLES

What this does: Tells Ubuntu where Java and Hadoop are installed.

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >> ~/.bashrc
echo 'export HADOOP_HOME=/usr/local/hadoop' >> ~/.bashrc
echo 'export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin' >> ~/.bashrc
source ~/.bashrc
```

STEP 3A — SET HADOOP'S JAVA PATH

What this does: Tells Hadoop where Java is installed.

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64' >> "$HADOOP_HOME/etc/hadoop/hadoop-env.sh"
hadoop version
```

STEP 4 — CONFIGURE PASSWORDLESS SSH

What this does: Allows Hadoop startup scripts to connect to localhost.

```
sudo systemctl enable --now ssh
mkdir -p ~/.ssh
chmod 700 ~/.ssh
[ -f ~/.ssh/id_rsa ] || ssh-keygen -t rsa -N "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh -o StrictHostKeyChecking=accept-new localhost
```

If SSH opens a remote shell, type **exit** to return to your original terminal.

STEP 5 — CONFIGURE core-site.xml

What this does: Sets the default HDFS address to localhost on port 9000.

```
cat > "$HADOOP_HOME/etc/hadoop/core-site.xml" <<'EOF'
<configuration>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
EOF
```

STEP 6 — CONFIGURE hdfs-site.xml

What this does: Sets one replica and defines local HDFS storage locations.

```
mkdir -p "$HADOOP_HOME/data/namenode"
mkdir -p "$HADOOP_HOME/data/datanode"

cat > "$HADOOP_HOME/etc/hadoop/hdfs-site.xml" <<'EOF'
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:///usr/local/hadoop/data/namenode</value>
    </property>
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:///usr/local/hadoop/data/datanode</value>
    </property>
</configuration>
EOF
```

STEP 7 — CONFIGURE MAPREDUCE

What this does: Configures MapReduce to use YARN.

```
cat > "$HADOOP_HOME/etc/hadoop/mapred-site.xml" <<'EOF'
<configuration>
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
EOF
```

STEP 8 — CONFIGURE YARN

What this does: Enables the shuffle service required by MapReduce.

```
cat > "$HADOOP_HOME/etc/hadoop/yarn-site.xml" <<'EOF'
<configuration>
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>
</configuration>
EOF
```

STEP 9 — FORMAT THE NAMENODE

What this does: Initializes HDFS metadata for a new filesystem. WARNING: only do this for a fresh setup; do not reformat an existing filesystem containing data.

```
hdfs namenode -format
```

STEP 10 — START HADOOP SERVICES

What this does: Starts HDFS and YARN, then lists running Java processes.

```
start-dfs.sh
start-yarn.sh
jps
```

Expected processes usually include NameNode, DataNode, ResourceManager, and NodeManager.

STEP 11 — CREATE A SAMPLE INPUT FILE

What this does: Creates a small text file for Word Count.

```
echo "hello hadoop hello spark" > ~/file.txt
```

STEP 12 — CREATE AN HDFS INPUT DIRECTORY

What this does: Creates /input in HDFS.

```
hdfs dfs -mkdir -p /input
```

STEP 13 — UPLOAD THE INPUT FILE

What this does: Copies the local file into HDFS.

```
hdfs dfs -put -f ~/file.txt /input/
```

STEP 14 — RUN MAPREDUCE WORD COUNT

What this does: Runs Hadoop's built-in Word Count example and saves results to /output.

```
hdfs dfs -rm -r -f /output
hadoop jar "$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar" wordcount /input /output
```

The output directory must not already exist. The first command removes a previous output directory so you can rerun the job.

STEP 15 — DISPLAY THE RESULTS

What this does: Prints the word counts stored in HDFS.

```
hdfs dfs -cat /output/part-r-00000
```

Expected output (word order may vary):

```
hadoop  1
hello   2
spark   1
```

RESULT

Hadoop processes the input using MapReduce and stores the word counts in HDFS.

Troubleshooting: If a command fails, stop at that step and read the error. Check the Java path if Hadoop cannot find Java, and test `ssh localhost` if Hadoop startup fails.